Figure 6.16 When using the GitHub Container Registry, you can store container images next to the source code.

NOTE Images published to GitHub Container Registry will have the same visibility as the related GitHub code repository. If no repository is associated with the image, it's private by default. Throughout the book I'll assume that all the images you build for Polar Bookshop are publicly accessible via the GitHub Container Registry. If that's not the case, you can go to the package's main page, select Package Settings from the sidebar menu, scroll to the bottom of the settings page, and make the package public by clicking the Change Visibility button.

Great job! So far you've built a Spring Boot application that exposes a REST API and interacts with a relational database; you wrote unit and integration tests for the application; you handled the database schema with Flyway so it was production-ready; and you ran everything within containers and dealt with image generation, Docker, Cloud Native Buildpacks, and vulnerability scanning. The next chapter will complete the first part of this cloud native journey toward production by diving deep into Kubernetes. But before moving on, take a break, congratulate yourself on what you have achieved so far, and perhaps celebrate with a beverage of your choice.

Polar Labs

Feel free to apply what you've learned in this chapter to Config Service.

1. Configure the Cloud Native Buildpacks integration and package the application as a container.
2. Update your Docker Compose file to run Config Service as a container.
3. Configure Catalog Service with the Config Service URL via the `SPRING_CLOUD_CONFIG_URI` environment variable, relying on the Docker built-in DNS.
4. Bootstrap a deployment pipeline for Config Service by implementing the workflow for the commit stage using GitHub Actions.